



ITMO UNIVERSITY

Встроенные системы

Мастер-класс 1

«Язык С во встроенных системах»

Сергей Быковский, доцент ф-та ПИиКТ
Алексей Гончаров, ассистент ф-та ПИиКТ

ITMO *re than a*
UNIVERSITY

План занятия

Теоретическая часть

- Чем embedded C отличается от desktop C?
- Модель памяти микроконтроллера (Flash, SRAM, периферия)
- Ключевое слово *volatile*
- Прямой доступ к памяти периферии (Memory-Mapped I/O)
- Организация программы: Бесконечный цикл в *main()*
- Обработчики прерываний (ISR)

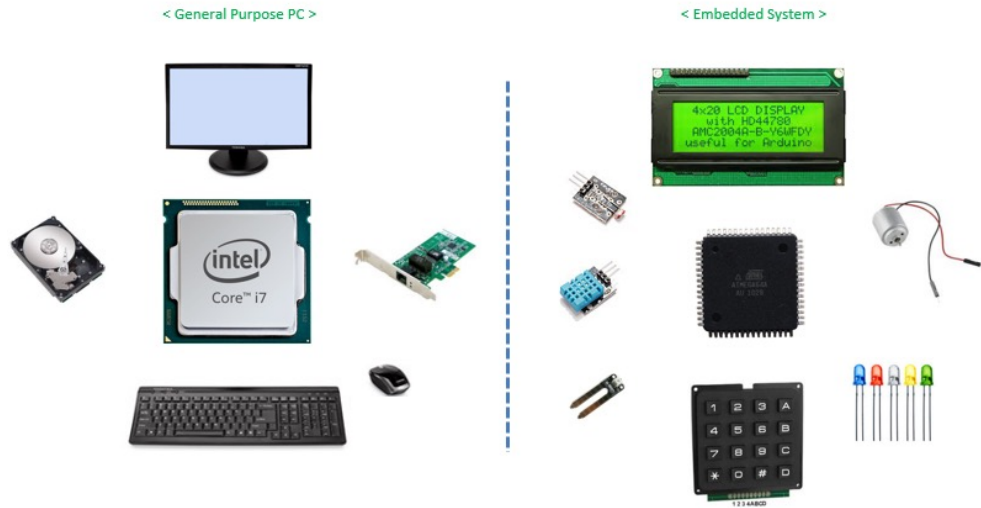
Практическая часть:

- Разбор примера кода для Wokwi
- Обсуждение задания

Введение: Мир встроенного Си

Чем Embedded C отличается?

- **Близость к железу:** Программа напрямую управляет регистрами.
- **Ограниченность ресурсов:** Мало памяти и вычислительных ресурсов.
- **Детерминированность и реальное время:** Код должен реагировать на события строго вовремя.
- **Нет выхода из *main()*:** Программа работает "вечно", управляя устройством.

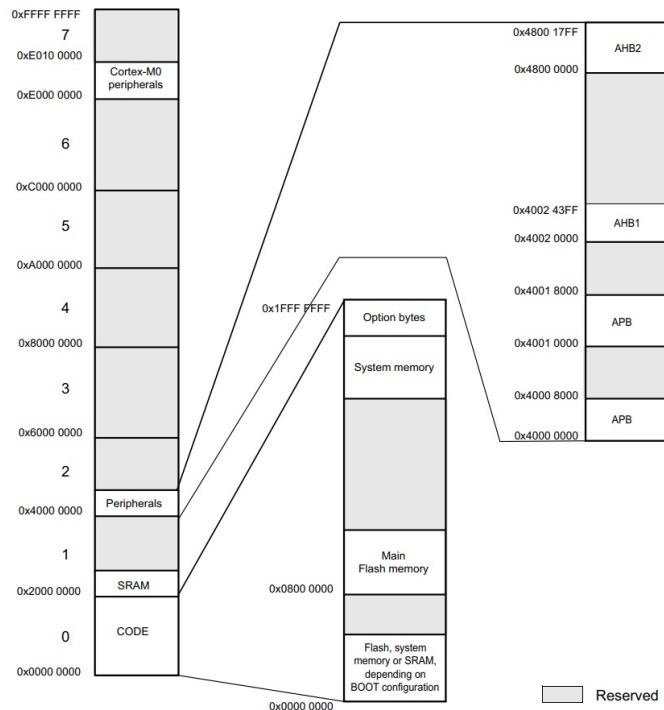


Модель памяти STM32

Память МК не однородна. Адреса заранее определены производителем.

- **Flash (0x0800 0000):** Хранит программу (.text) и константы (.rodata).
- **SRAM (0x2000 0000):** Хранит переменные (.data, .bss), стек (stack), кучу (heap).
- **Периферийные регистры (0x4000 0000 - 0x6000 0000):** Запись/чтение по этим адресам влияет на состояние ножек МК и периферии.

Вывод: адрес определяет функцию ячейки памяти.



Memory-Mapped I/O (Отображение периферии на память)

Концепция: Каждый регистр периферии (например, регистр включения светодиода) имеет свой уникальный адрес в памяти.

Как работать? Через указатели на структуры, описывающие набор регистров устройства.

```
#define GPIOA_BASE 0x50000000
typedef struct { ... } GPIO_TypeDef;
#define GPIOA ((GPIO_TypeDef*) GPIOA_BASE) // Указатель на структуру по адресу GPIOA_BASE

GPIOA->ODR |= (1 << 5); // Запись в регистр данных порта A
```

Ключевое слово *volatile*

Проблема: Компилятор оптимизирует код. Если он видит, что переменная не меняется в коде, он может использовать ее кешированное значение.

Решение: *volatile*

- Сообщает компилятору, что переменная может измениться извне (например, аппаратным прерыванием).
- Заставляет компилятор всегда читать ее значение из памяти и всегда записывать новое значение обратно.

Где использовать?

1. Переменные, используемые в обработчиках прерываний.
2. Указатели на периферийные регистры.

Организация программы: Бесконечный цикл

В обычной программе на ПК: *main()* запускается, работает и завершается (возвращает значение в ОС).

В программе для МК: *main()* не должен завершаться. Устройство должно работать постоянно.

```
int main(void) {  
    hardware_init(); // Инициализация всей периферии  
  
    while (1) {      // Бесконечный цикл  
        checkButtons();  
        processData();  
        updateDisplay();  
        // Код здесь выполняется снова и снова...  
    }  
    // Сюда мы никогда не попадем!  
    return 0;  
}
```

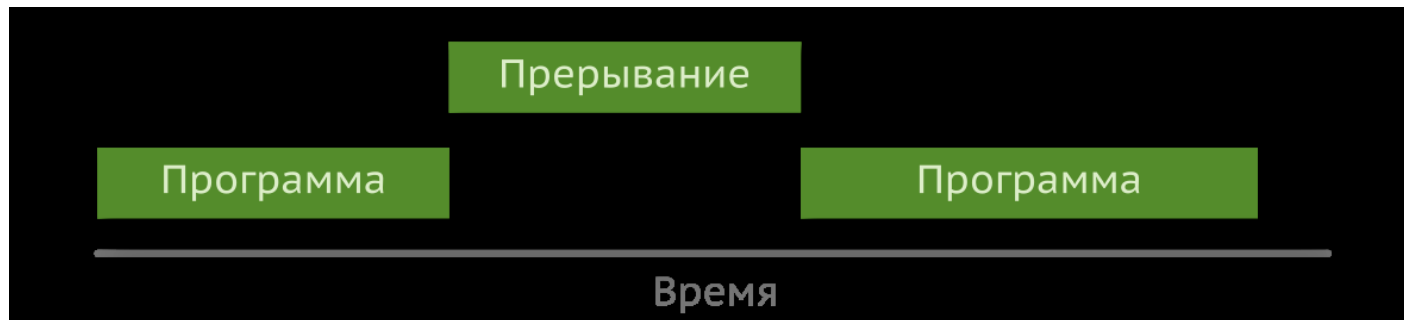
Прерывания и их обработчики (ISR)

Что такое прерывание? Событие, которое заставляет немедленно отложить текущую задачу и выполнить специальную функцию-обработчик.

Виды событий: Таймер, нажатие кнопки (внешнее прерывание), получение данных по UART.

Обработчик прерывания (ISR - Interrupt Service Routine):

- Короткая функция, которая должна быстро обработать событие.
- **Важно:** Нежелательно делать в ISR долгих вычислений.



Пример обработчика прерывания

В нашем коде: Обработчик для системного таймера (*SysTick*), который вызывается каждую 1 мс

```
volatile uint32_t tickCount = 0;

void osSystickHandler(void) { // Эта функция вызывается аппаратно!
    tickCount++; // Инкрементим счетчик
}
```

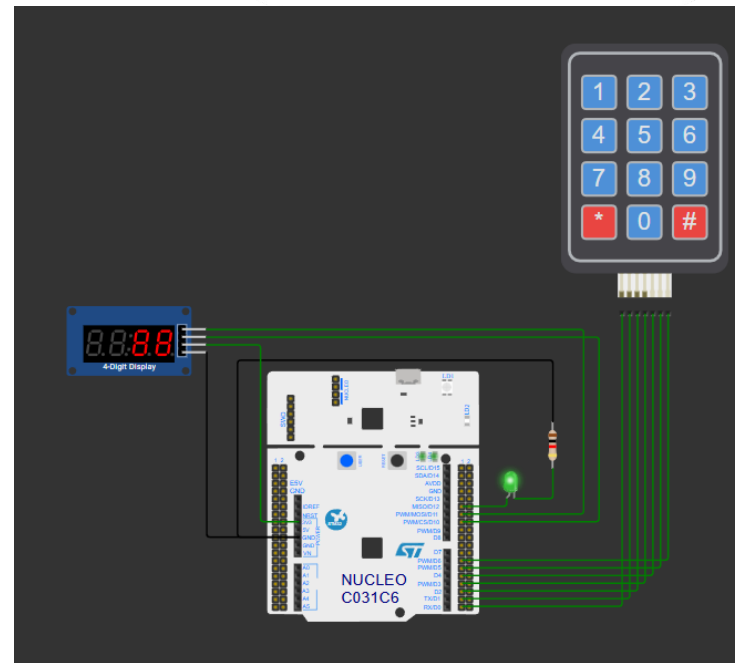
Как это работает? Аппаратура таймера генерирует прерывание, прерывается выполнение *main()*, переход на адрес *osSystickHandler*, выполнение и возвращение обратно в *main()*.

Обзор примера кода для Wokwi (Часть 1)

Цель программы: Мигать светодиодом (PA5) и выводить сообщение в UART каждые 2 секунды, каждую секунду обновлять 4-значный дисплей и раз в 100 мс проверять нажатую клавишу.

Структура:

- **Определение адресов:** `RCC_BASE`, `GPIOA_BASE` и т.д.
- **Структуры регистров:** `RCC_TypeDef`, `GPIO_TypeDef` – "карта" регистров периферии.
- **Указатели на периферию:** `RCC`, `GPIOA` – доступ к устройствам через эти структуры.
- **Битовые маски:** `USART_CR1_UE` – для удобной работы с битами регистров.



Обзор примера кода для Wokwi (Часть 2)

Функции инициализации:

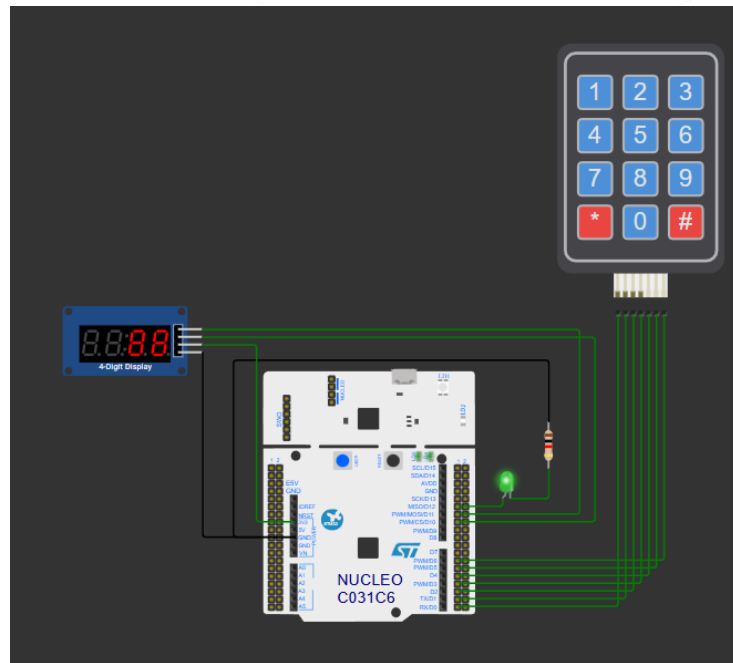
- *initGPIO()*: Включает тактирование порта A, B и настраивает ножку PA5 на выход.
- *initUSART2()*: Настраивает USART для передачи данных.
- *initSysTick()*: Запускает системный таймер.
- *initKeyboard()*: Инициализация клавиатуры
- *tm1637_init()*: подключение 4-значного дисплея.

Функция *_write()*: Переопределенная низкоуровневая функция для работы *printf* через UART.

Главный цикл:

Бесконечно вызывает *checkTickCount()*, *scanKeyboard()*

и *tm1637_update()*.



Анализ функции *checkTickCount()*

```
void checkTickCount() {  
    if ((tickCount % 2000) == 0) { // Проверяем, набегало ли 2000 мс (2 сек)  
        GPIOA->ODR ^= (1 << 5); // Инвертируем состояние пина PA5 (XOR)  
        printf("tickCount = %d!\n", tickCount++); // Выводим в симулятор  
    }  
}
```

Важно: Переменная *tickCount* изменяется в прерывании, а читается в главном цикле.

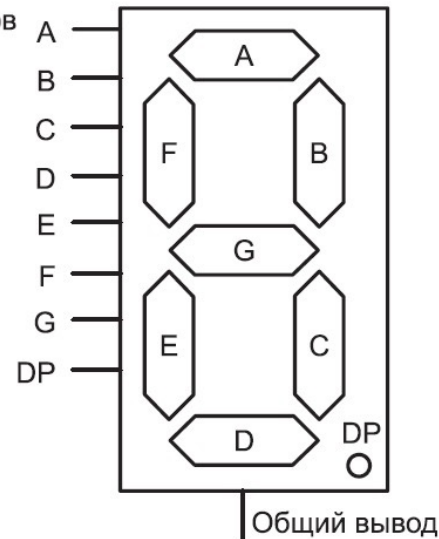
Это классический случай, где она должна быть объявлена как *volatile uint32_t tickCount*;

Семисегментный индикатор

Предназначен для отображения цифр от 0 до 9, а также некоторых букв и символов.

Индикатор через сопротивление (R) подключается к портам ввода-вывода микроконтроллера.

Выводы сегментов индикатора



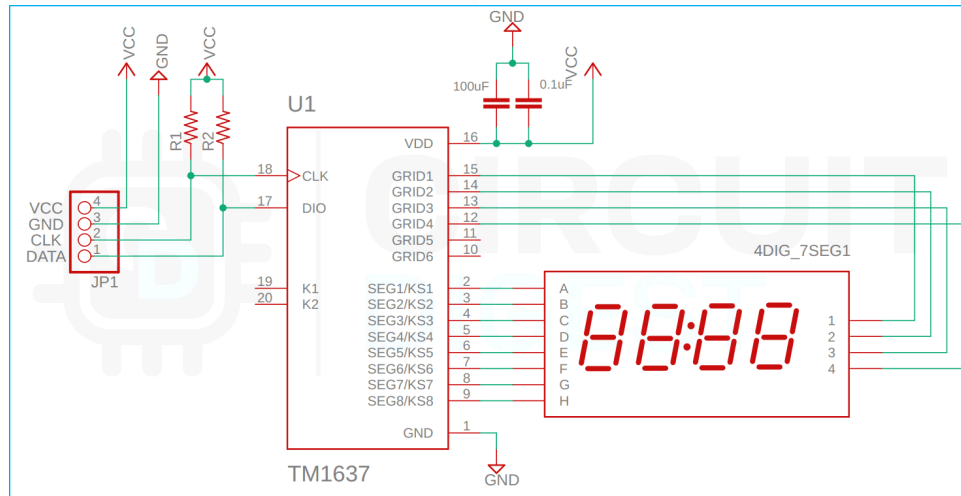
Инструкция:

<https://storage.googleapis.com/media.amperka.com/products/7-segment-led/media/sc56-11gwa.pdf>

Работа с TM1637

TM1637 – внешнее устройство с последовательным интерфейсом, которое:

- Имеет собственную **систему команд** (инициализация, запись данных, управление яркостью)
- Требует **строгого соблюдения временных диаграмм**
- Демонстрирует принцип **разделения обязанностей** (микроконтроллер обрабатывает логику, TM1637 управляет индикацией)



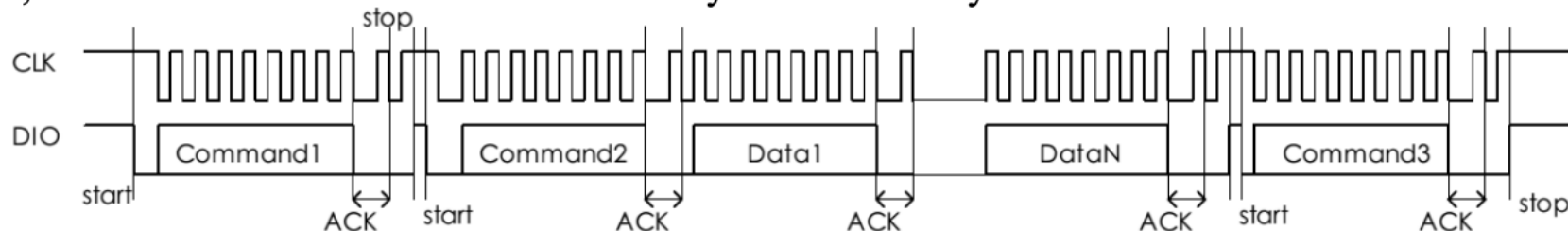
Инструкция:

<https://static.chipdip.ru/lib/110/DOC001110878.pdf>

В нашем примере позволяет объединить четыре семисегментных индикатора и управлять ими через 2 GPIO (CLK и DATA) + VCC + GND

Инструкция: <https://static.chipdip.ru/lib/110/DOC001110878.pdf>

2, SRAM data write address is automatically incremented by 1 mode



Command1: Set Data

Command2: Set Address

Data1 ~ N: display data transmission

Command3: Control Display

Работа с матричной клавиатурой

Чтобы микроконтроллер определил, какая кнопка нажата, ему сначала необходимо поочередно установить каждый из четырех столбцов (выводы 1–4) в низкий или высокий уровень, а затем опросить состояния четырёх строк (выводы 5–8). В зависимости от состояния столбцов микроконтроллер может определить, какая кнопка нажата.

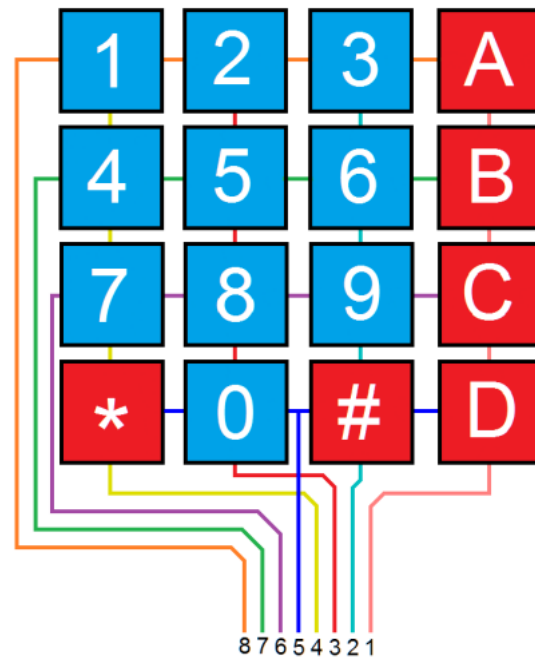


Figure 1: Matrix Keypad Connections

1. Всегда инициализируйте периферию перед использованием.
2. Помните о *volatile* для глобальных переменных и указателей на регистры.
3. Делайте обработчики прерываний (ISR) короткими. Переносите тяжелые вычисления в главный цикл.
4. Избегайте блокирующих задержек. Используйте таймеры и флаги для отсчета времени.
5. Внимательно работайте с битами: Используйте операции AND (&), OR (|), XOR (^) и сдвиги (<<, >>) для управления отдельными битами регистров, не задевая остальные.

- Программирование МК на Си — это прямой диалог с железом через память.
- Понимание модели памяти и механизма прерываний — ключевой навык.
- Ключевые слова: *volatile*, *Memory-Mapped I/O*, *ISR*, бесконечный цикл.
- Симуляторы (Wokwi) — отличный полигон для обучения без риска "спалить" железо.

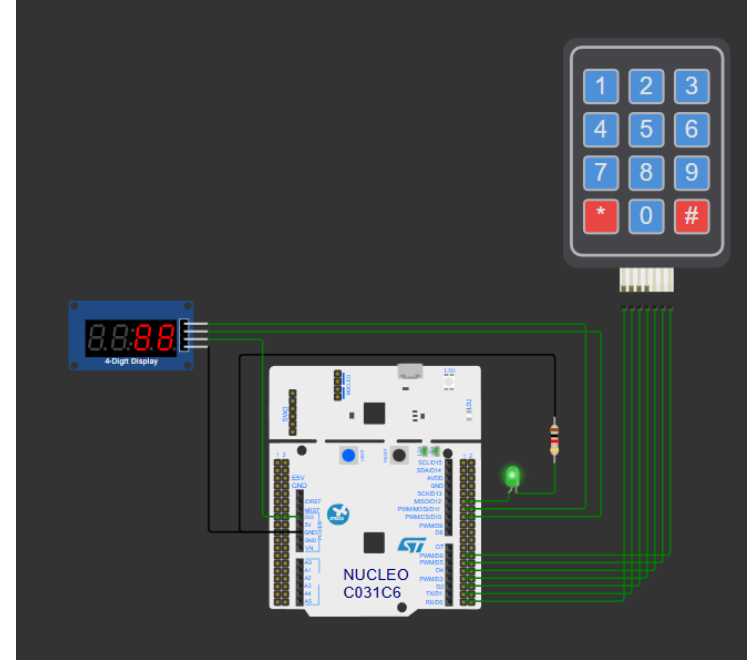
Порядок выполнения задания и защиты

- Выполнить задание для получение варианта. У всех одинаковое.
- Получить вариант и выполнить задание по варианту.
- Написать тест по мастер классу в начале следующего занятия
- Продемонстрировать работоспособность проекта преподавателю.
- Сделать отчет и согласовать его с преподавателем.
- Загрузить согласованный отчет в форму:

<https://forms.yandex.ru/u/6a97414beb6146284a05cb77>

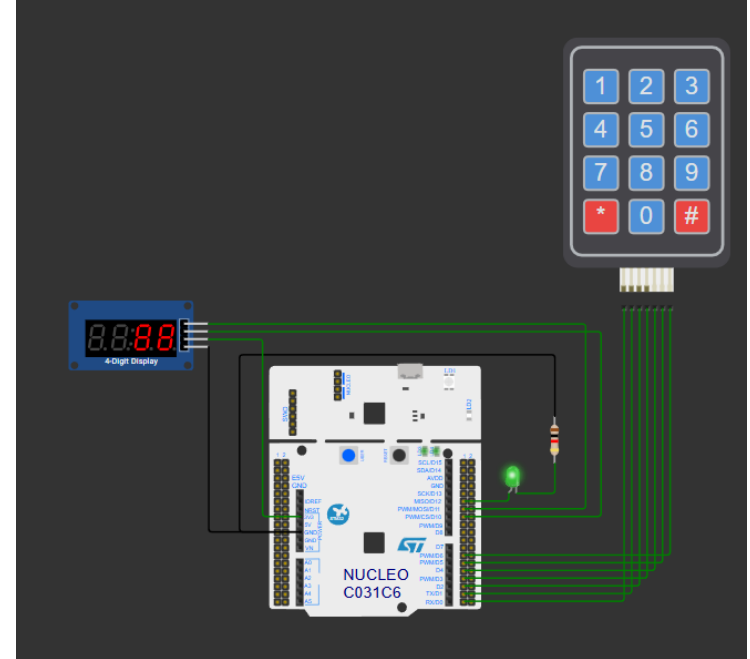
Задание для получения варианта

- Разработать таймер. Количество секунд задается с помощью клавиатуры и отображается на дисплее. Когда таймер досчитает до 0, то должен загораться светодиод.



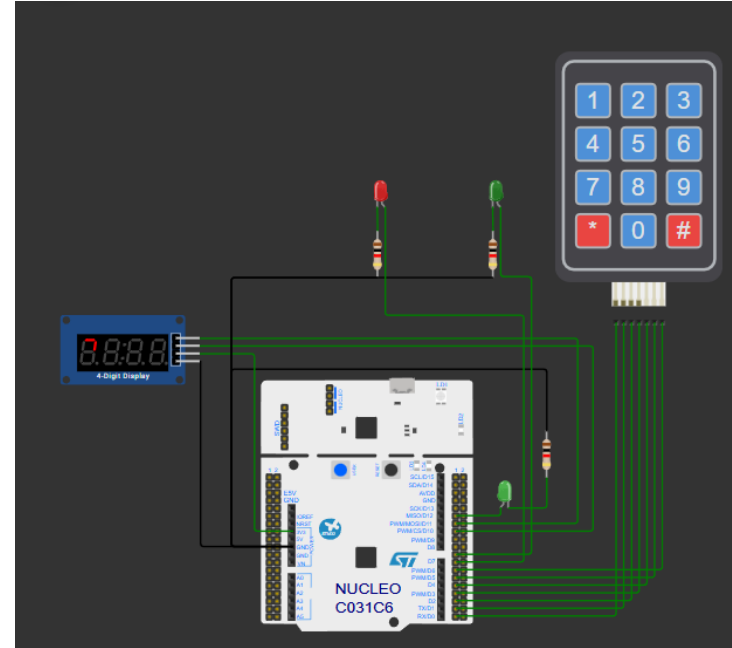
Задание | Вариант 1

- Разработать калькулятор. В качестве операндов использовать два числа, каждое из которых от 0 до 9. Поддерживаемые операции: +, -, *, /. Выбор операции осуществляется посредством клавиши *. Результат вычисления появляется на семисегментном индикаторе после нажатия на #.



Задание | Вариант 2

- Разработать систему управления доступом с использованием 4-разрядного семисегментного индикатора, мембранной клавиатуры и светодиодов для имитации процесса ввода кода из 4-ёх цифр.



Пособие и примеры программ

- Документация по симулятору: <https://docs.wokwi.com/>
- Справочное руководство nucleo-c031c6:
https://www.st.com/resource/en/reference_manual/rm0091-stm32f0x1stm32f0x2stm32f0x8-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- Пример программы: <https://wokwi.com/projects/441248357913176065>